AEGIS-C2R

Deterministic C-to-Safe-Rust Translation Prototype

Engineering, Validation, Reproduction and Roadmap Manual

Release: 0.1.0

Profile: S0-DEFINED-I32

Primary implementation: strict C11

Aegis integration: DXL + DXS + DXASM, clean 2.4.0 C stage-zero bootstrap

License: MIT

Validation date: 24 August 2026

Purpose: executable portfolio proof and engineering prototype, not a PhD thesis and not a claim that arbitrary C-to-Rust migration is solved.

2. Executive Summary

Aegis-C2R 0.1.0 is a working deterministic translator for a deliberately bounded C profile. Its job is not to accept everything. Its job is to make every accepted construct explicit, lower it to safe Rust in one repeatable way, and attach evidence that can be reproduced without trusting a language model or a hidden service.

The translator is written in strict C11. It performs its own bounded lexing, parsing, semantic checks, canonical AST serialization, Rust generation, SHA-256 binding and proof-record emission. Generated Rust starts with #![forbid(unsafe_code)]. Arithmetic operations that correspond to undefined C boundaries use Rust checked operations so the generated program fails explicitly rather than pretending that undefined C behavior has a unique meaning.

The project integrates the clean Aegis three-language path. The translator emits tiny DXL, DXS and DXASM certificate programs from the same AST metrics. The DXS and DXASM programs are compiled to DXB3, audited and executed by the vendored Aegis 2.4.0 toolchain bootstrapped from strict C. Repeated compilation is compared byte-for-byte. All three paths must produce the same certificate.

The release gate covers positive programs, deterministic rejection, compiler disagreement checks, canonicalization, SHA-256 recomputation, declared resource boundaries, 512 generated semantic vectors, 256 source mutations, strict GCC/Clang builds, GCC analyzer and ASan/UBSan. The one unavailable gate on this build host is rustc compilation. The scripts mark that gate SKIP and automatically run it on a host where rustc exists.

- This is evidence for a real implemented subset, not evidence for all C.
- No Canonical, C2Rust or Cpp2Rust source is copied or vendored.
- The full package contains source, compiled C/Aegis binaries, documentation and validation evidence.

3. Why Build This Now?

Canonical publicly announced on 19 August 2026 that it is funding a three-year PhD with the University of Bristol around automated C-to-Rust translation for Ubuntu. The announced direction combines repository partitioning, translation, validation and repair. That validates the importance of the problem, but it does not make a small independent implementation pointless. A focused prototype can demonstrate engineering skill today while the larger research program develops over years.

The correct response is not to imitate a future system or to claim to have solved its full research problem. Aegis-C2R chooses a smaller target that can be executed and falsified now. The first release handles scalar int code, structured control flow, simple calls and a tightly controlled printf/assert interface. Every unsupported category fails closed.

This design also avoids the common portfolio trap of presenting architecture diagrams without a program. The release contains an actual parser, AST, emitter, test harness, mutation gate, sanitizer gate and Aegis cross-language certificate path. The documentation is therefore downstream of executable behavior rather than a substitute for it.

The competitive lesson from existing translators is that breadth and safety are different axes. C2Rust has broad C99-oriented migration and may intentionally emit unsafe Rust. Cpp2Rust tackles much harder C++ pointer semantics using a safe runtime abstraction. Aegis-C2R 0.1.0 occupies a different point: narrow admitted syntax, no unsafe block in generated output, exact deterministic artifacts and explicit evidence boundaries.

- Goal: demonstrate a deterministic translation core before expanding breadth.
- Non-goal: claim that Ubuntu/AppArmor/snap-confine can be migrated by this release.

4. Claims, Non-Claims and Clean-Room Boundary

The most important credibility feature is the claim boundary. Aegis-C2R claims that the implemented S0-DEFINED-I32 syntax is parsed and lowered according to the documented rules, that repeated translation is byte-stable, and that the supplied test evidence passed on the stated host. It does not claim formal proof of arbitrary C equivalence, complete C semantics, a complete Rust compiler front end, or production maturity.

Only public documentation and published descriptions of other tools were consulted. Their implementation source was not copied into this repository. The translator architecture, AST, diagnostic scheme, proof JSON and Aegis certificate integration are implemented in this project. The only vendored project source is the Aegis Language Suite 2.4.0 source-bootstrap snapshot from the same Aegis lineage, with the MIT license preserved.

This distinction matters professionally. A convincing portfolio artifact should let another engineer inspect what is original, what is reused under license, what is only a comparison reference, and what has actually been executed. The THIRD_PARTY.md and RESEARCH_NOTES.md files make this separation machine-visible in the source tree.

The phrase deterministic 1:1 is used here in a precise restricted sense: identical admitted source bytes produce identical AST/Rust/proof bytes; formatting-equivalent source produces an identical canonical AST and Rust while retaining a different source hash; and the declared semantic vectors are compared against independent C compiler outputs. It is not a claim that every possible C program has a one-to-one safe Rust representation.

- Shipped claims are tied to tests.
- Planned work is labeled roadmap, not current functionality.
- Unavailable validation tools are SKIP, never silently PASS.

5. System Architecture

The core pipeline is deliberately short. The input C file is read with a one-megabyte limit. A deterministic lexer removes whitespace, comments and two permitted include lines. A recursive-descent parser constructs an arena-allocated AST and simultaneously applies profile constraints. Semantic failures stop the pipeline before output is emitted.

After successful parsing, one AST feeds all output paths. The canonical AST serializer emits a location-independent textual form. The Rust emitter lowers the AST using fixed templates and checked integer helpers. SHA-256 is calculated over the exact source bytes, AST bytes and Rust bytes. A deterministic JSON record binds these identities to AST metrics and feature flags.

The Aegis proof capsule is generated from the same function, statement and expression counts. DXL, DXS and DXASM therefore act as three independent representations of the structural certificate. The DXS and DXASM paths converge on the verified DXB3 runtime boundary, where the Aegis auditor recomputes runtime constraints before execution.

Host GCC and Clang are validation oracles, not translation dependencies. This separation is intentional: the translator result does not vary because a different host compiler pretty-printed or optimized a source file. The host compilers are used to confirm that positive fixtures are legal C and to compare their runtime output.

```
C -> lexer -> parser/profile -> AST -> Rust
                          |       -> canonical AST
                          |       -> proof JSON
                          +       -> DXL / DXS / DXASM certificate
GCC + Clang C binaries ---------------------- validation oracles
```

6. Aegis 3-in-1 Integration

The Aegis language suite contributes three complementary layers. DXL is the typed systems-oriented language. DXS is a deliberately small deterministic stack language. DXASM is an explicit assembly notation for the shared bytecode model. In the clean 2.4.0 source-bootstrap release, a strict-C stage-zero DXL compiler breaks the bootstrap cycle and builds the DXL-authored DXS compiler, DXASM assembler, auditor and VM.

Aegis-C2R does not use Aegis as decoration. For each translated program it can emit three certificate programs. The DXL file performs typed arithmetic, asserts the expected value and prints it. The DXS file computes the same value on the verified stack machine. The DXASM file expresses the same operations instruction by instruction.

The release test compiles DXS twice and requires byte-identical DXB3. It assembles DXASM twice and requires byte-identical DXB3. It executes repeated VM runs and compares proof bytes. Finally, the last printed result from DXL, DXS and DXASM must equal the JSON certificate. This catches accidental disagreement between the translator proof emitters and the Aegis execution paths.

The certificate is intentionally described as structural. A 32-bit number cannot prove whole-program semantic equivalence. Its value is that it makes cross-language disagreement visible and reproducible while the stronger semantic evidence comes from AST/lowering contracts and differential execution.

- DXL: typed certificate oracle.
- DXS: compact deterministic stack program.
- DXASM: explicit instruction-level witness.
- DXB3 audit executes before VM acceptance.

7. C Stage-Zero Bootstrap and Trust

The requested Aegis base is the clean source-bootstrap path rather than a binary-only toolchain. The vendored version includes the strict-C stage-zero compiler source. On a clean host, make bootstrap builds that compiler with the host C compiler; make toolchain then uses it to turn the readable DXL tool sources into executable Aegis tools.

This structure narrows the trust story. The bootstrap is not magic: a host C compiler is still trusted to compile the stage-zero source. But the readable DXL implementations can then be exercised against the reference behavior, and the Aegis release itself tests repeated DXS/DXASM byte identity. The release does not claim complete self-hosting of the entire DXL compiler frontend.

During Aegis-C2R final validation, the vendored Aegis 2.4.0 make check completed 11 PASS / 0 FAIL: DXL check/run, DXS compile/audit/execute, DXASM assemble/execute, repeated DXS compilation with byte identity and repeated DXASM assembly with byte identity. Brand-hygiene validation also passed.

The full AIO package includes compiled Aegis tools for the tested Linux x86-64 environment. The clean Git source package excludes build directories so a reviewer can reproduce the bootstrap instead of trusting the included binaries.

- Trust root is explicit rather than hidden.
- Compiled artifacts are convenience, not the source of truth.
- Reproduction is part of the deliverable.

8. The S0-DEFINED-I32 Profile

S0-DEFINED-I32 is intentionally more restrictive than C99. A translation unit contains int-returning functions, with main final. Parameters and locals are signed 32-bit int. User functions must appear before use; recursion and forward calls are rejected. Local declarations must form a prefix of the outer function body and every local has an initializer.

Expressions support decimal literals, variables, calls, unary minus/not, arithmetic, comparisons and short-circuit logical operators. Statements support declaration, assignment, return, function-call statement, if/else, while, blocks, printf("%d\n", expr) and assert(expr).

Pointers, arrays, structs, unions, floating point, unsigned types, globals, macros, typedefs, varargs, for, switch, goto, function pointers, concurrency, atomics, volatile and arbitrary libc/FFI calls are rejected. This is not a weakness hidden in a footnote; it is the central mechanism that keeps release 0.1.0 auditable.

The parser also enforces name rules needed for stable lowering. A new local is not visible in its own initializer. Parameters and locals cannot shadow known callable names or the printf/assert builtins. Generated Rust prefixes user names with c2r_, avoiding collisions with Rust keywords without changing C name lookup semantics.

- Input limit: 1 MiB.
- Functions: max 63. Parameters: max 16. Locals: max 256.
- AST nodes: max 65,535. Nesting: max 64.

9. Lexer and Preprocessor Gate

The lexer is deliberately self-contained and deterministic. It recognizes ASCII identifiers, signed-profile decimal literals, punctuation and the exact operators required by S0. Line and block comments are skipped. Newlines update explicit line/column counters so diagnostics remain stable across runs.

The preprocessor is not invoked. Instead, lines beginning with # are normalized by removing whitespace and compared against two accepted include forms: stdio.h and assert.h. Every other directive fails with a profile diagnostic. This means a build environment cannot change translation semantics by defining a macro differently or selecting a different system header.

The decision trades compatibility for reproducibility. Full C migration will eventually need a preprocessing capture mode or a compilation database, but that belongs to a later repository-scale profile where exact macro expansion and header identities are recorded. In S0, rejecting that complexity is safer than silently importing it.

Integer tokenization accepts only values that fit the positive i32 literal range. Unary minus is represented separately in the AST. The boundary test proves 2,147,483,647 is accepted and 2,147,483,648 is rejected. The current profile therefore does not encode the lexical special case for spelling INT_MIN directly; that limitation is explicit.

- Unknown bytes fail closed.
- Unterminated block comments produce a deterministic lexical error.
- Unsupported preprocessing never reaches the parser.

10. Parser and Canonical AST

The parser is a handwritten recursive-descent parser. The expression stack follows C precedence from unary operations through multiplicative, additive, relational, equality, logical AND and logical OR. Binary AST nodes therefore record the semantic tree rather than the original parentheses or whitespace.

Statements are linked AST nodes inside explicit block nodes. Every allocation passes through a counted arena. The program records AST node count, expression count, statement count, maximum nesting and feature flags. At the end of parsing, arena allocation count and AST accounting must agree or translation stops with an internal invariant error.

The canonical AST serializer deliberately omits line and column information. This means two source files that differ only in whitespace/comments can produce identical AST and Rust bytes. The validation suite includes basic.c and basic_whitespace.c to prove this. Their source SHA-256 values differ, while AST and Rust SHA-256 values match.

This separation is useful in provenance work: source identity answers “which exact file did we translate?”, while AST identity answers “which admitted program did the front end understand?” Both are retained instead of choosing one and losing the other.

```
AEGIS-C2R-AST/1
profile S0-DEFINED-I32
metrics functions=... statements=... expressions=...
function name params=...
    (block ...)
end-function
```

11. Semantic Checks Before Emission

The parser performs semantic checks at the moment enough information exists to make them deterministic. Variable reads and assignments require a known parameter/local. Function calls require a previously declared function with an exact argument count. Direct recursion is rejected. Main must have no parameters and must be final.

Local declaration handling was hardened during finalization. The initializer is parsed before the new local is published to the symbol table, so int x = x is rejected instead of producing a Rust reference to a binding that does not yet exist. Callable-shadow checks also prevent a local or parameter from changing how a function-looking expression should be interpreted.

Every function must end with an explicit top-level return. This restriction is stronger than C but makes the initial lowering total and easy to audit. More general control-flow return analysis can be added later as a separate verified feature rather than by weakening the current invariant.

Diagnostic categories are stable numeric families: I/O, lexical, parse, profile, semantic, emission and internal errors. Negative fixtures run twice, compare exit codes and compare stderr byte-for-byte. A source/profile failure must not produce the requested Rust or proof output.

- Fail closed is a feature, not a parser failure.
- Every supported feature has a positive fixture.
- Every major unsupported category has a negative fixture.

12. Safe Rust Lowering

The Rust emitter is template-driven from the AST. It does not splice arbitrary C text into generated Rust. User functions become c2r_name, variables become c2r_name, and C main becomes c2r_main_impl plus a small Rust main wrapper.

The first source-level safety guard is #![forbid(unsafe_code)]. A deterministic release guard verifies that this attribute exists and that no other unsafe token appears. This is not a replacement for rustc, but it prevents the emitter from silently introducing an unsafe block while the optional compile gate provides compiler validation when Rust is installed.

if and while conditions convert the i32 C truth value into a Rust boolean by comparing against zero. Relational/equality operators convert Rust bool back to i32 using as i32. Logical AND/OR preserve short-circuiting: the right-hand Rust expression is evaluated only if C would evaluate its right-hand operand.

printf("%d\n", expr) becomes println!("{}", expr). assert(expr) becomes a Rust assertion against zero. These mappings are profile-specific. They are not intended to generalize to arbitrary printf formats or C runtime behavior.

```
#![forbid(unsafe_code)]
fn c2r_add(c2r_a: i32, c2r_b: i32) -> i32 {
    return ae_add(c2r_a, c2r_b);
}
fn main() { std::process::exit(c2r_main_impl()); }
```

13. Arithmetic and the C Undefined-Behavior Boundary

C signed overflow is undefined behavior. A translation system must not quietly pick wraparound semantics and then advertise that choice as universal C equivalence. Aegis-C2R instead defines its semantic claim over executions where the admitted C program remains defined.

Generated Rust uses checked_add, checked_sub, checked_mul, checked_div, checked_rem and checked_neg. For defined C executions these helpers return the same i32 result. If an execution reaches signed overflow, divide-by-zero, INT_MIN divided by -1 or negation overflow, Rust terminates explicitly. That behavior is a safety fence, not an assertion that the original C program had a defined matching outcome.

The same reasoning applies to other future C features. Pointer invalidity, uninitialized reads, strict aliasing, lifetime violations and out-of-bounds operations must be modeled or rejected; they cannot be solved by a textual syntax converter. The roadmap therefore delays pointer support until an explicit region/provenance abstraction and corresponding tests exist.

This boundary makes the project easier to defend technically. A reviewer can challenge an individual lowering rule rather than discovering that “equivalent” was being used to cover undefined source semantics.

- Defined input path -> same mathematical i32 operation.
- Undefined source operation -> outside equivalence claim and checked failure in Rust.
- No -fwrapv assumption is built into the translator.

14. Determinism Model

Determinism is tested at multiple layers. The same source is translated twice and Rust, AST, JSON proof, DXL, DXS and DXASM outputs are compared byte-for-byte. The JSON proof contains no timestamp, host name, random nonce or absolute path, removing common sources of accidental release drift.

The optimized GCC translator itself is built twice from a clean tree and compared byte-for-byte during release sealing. This is a useful reproducibility check on the tested host, although it is not a promise that arbitrary compiler versions or operating systems will emit the same native executable.

Canonicalization adds a second kind of determinism. Whitespace/comment variation changes exact source identity but not the AST/Rust identity. The proof record therefore binds both levels. This prevents a semantic cache from confusing two different source artifacts while still allowing a reviewer to observe canonical equivalence.

The 256-case mutation gate gives determinism an adversarial component. Each deterministic byte mutation is executed twice. If accepted, all outputs must match. If rejected, exit status and diagnostic bytes must match. In the final run, 11 mutations remained valid and 245 were rejected; all 256 were repeat-stable.

- Same bytes -> same products.
- Same canonical program -> same AST/Rust.
- Different source bytes remain distinguishable by SHA-256.

15. Proof JSON and SHA-256 Binding

The proof JSON is intentionally small enough to inspect without a specialized database. It identifies the schema, tool/version/profile, success status, SHA-256 of exact source, SHA-256 of canonical AST, SHA-256 of Rust output, AST metrics, feature booleans and the structural certificate.

The release includes an independent SHA-256 recomputation gate using Python hashlib. This caught category of risk is important because the translator contains its own small SHA-256 implementation; testing only self-consistency would not establish that its digest function matches the standard algorithm.

Hashing is identity evidence, not semantic evidence. A SHA-256 match tells a reviewer that the bytes are identical to a committed artifact. It does not tell them that the artifact is correct. The release therefore keeps hashing separate from compiler/runtime comparison and from Aegis certificate execution.

Because the proof record is deterministic, it can be version-controlled, attached to a CI artifact or compared during a later rebuild. A future release can sign a manifest without changing the underlying source/AST/Rust identity model.

```
{
  "schema": "AEGIS-C2R-PROOF/1",
  "profile": "S0-DEFINED-I32",
  "source_sha256": "...",
  "ast_sha256": "...",
  "rust_sha256": "...",
  "certificate": 50368527
}
```

16. Aegis Structural Certificate

The structural certificate packs three bounded metrics into one deterministic integer: functions multiplied by 16,777,216, statements multiplied by 4,096, plus expressions. Under the S0 declared limits the result stays inside a signed 32-bit range.

This calculation is simple on purpose. A certificate used as a cross-language canary should be easier to audit than the subsystem it is checking. The value is emitted into the JSON and also used as the expected assertion value in DXL, DXS and DXASM proof programs.

The DXS and DXASM programs duplicate the computed value, compare it to the expected certificate, assert the boolean, then print the retained value. Their shared DXB3 verifier checks the program before VM execution. DXL computes the same formula using typed i64 arithmetic and prints it after assertion.

If the translator metrics, one proof emitter or an Aegis runtime path diverges, the consensus gate fails. This is not a cryptographic commitment by itself; the SHA-256 identities provide that role. The certificate instead creates a cheap executable agreement check across three language surfaces.

```
certificate = functions * 16,777,216
            + statements * 4,096
            + expressions
```

17. Positive and Differential Tests

The hand-written positive corpus covers arithmetic, function calls, nested calls, branching, loops, precedence, negative arithmetic, comments/include normalization and short-circuit logic. Each file is compiled with strict GCC and strict Clang. Their runtime stdout must match before the translation result is trusted as the fixture reference.

Every positive source is then translated twice. Rust, canonical AST, proof JSON and all three Aegis certificate sources are compared byte-for-byte. The AST command-line output is also compared with the AST artifact emitted by translate, preventing drift between command paths.

The generated semantic corpus adds 512 deterministic vectors across four translation units. The vectors cover addition/multiplication, subtraction/division with C truncation semantics, remainder including negative values, comparisons, logical combinations, unary negation and logical not. GCC and Clang outputs are compared to a separately generated expected-output file.

When rustc is available, the same test script compiles every emitted Rust unit with warnings denied and compares its stdout with the GCC reference. This is intentionally an automatic extension of the normal gate rather than a separate manual recipe.

- Hand-written tests exercise structure.
- Generated vectors exercise value combinations.
- Two C compilers reduce dependence on one frontend.

18. Negative Testing and Fail-Closed Behavior

Negative tests are first-class release artifacts. They cover unsupported printf formats, literal divide-by-zero, floating point, forward calls, code after main, globals, late and nested declarations, macros, main parameters, malformed syntax, missing final return, absent main, pointers, recursion, unknown variables, for loops, unsupported includes, self-initializers and callable shadowing.

Each negative source is translated twice. A successful exit is a test failure. The diagnostic must contain an AE#### code. Both runs must return the same status and produce byte-identical stderr. Requested Rust/proof output files must not exist after the source/profile rejection.

This matters because “translator supports X” is only half of a deterministic contract. For every unsupported construct the tool must make the boundary visible. Silent approximation, guessed lowering or acceptance followed by malformed Rust would be worse than rejection.

The diagnostic family also helps future development. When a new feature is added, the corresponding negative fixture can be promoted or split into a positive case while other rejection behavior remains stable. That makes profile growth reviewable rather than ad hoc.

- Positive correctness and negative correctness are both required.
- No recovery mode silently edits the source.
- The trusted core has no LLM or network fallback.

19. Boundary and Mutation Testing

The declared numeric limits are executed at their edges. The final boundary matrix checks maximum i32 positive literal and plus-one rejection, 63-function acceptance and 64-function rejection, 16 parameters and 17 rejection, 256 locals and 257 rejection, nesting depth 64 and 65 rejection, plus rejection of input over one MiB. All eleven checks passed.

Boundary tests prevent a common documentation failure: declaring a maximum without testing the exact transition from allowed to rejected. They also exercise parser accounting paths that small examples never reach.

The deterministic mutation gate starts from a valid multi-function program and changes one non-newline byte in 256 seeded trials. The replacement alphabet includes punctuation, operators, digits, spaces and hostile characters. Each mutant is processed twice. Accepted mutants must reproduce all outputs; rejected mutants must reproduce status and diagnostic bytes.

The final mutation result was 256/256 repeat-stable: 11 accepted, 245 rejected. The ratio itself is not a security metric. Its purpose is to demonstrate that both parser success and parser failure remain deterministic under small adversarial perturbations.

- Boundary matrix: 11/11.
- Mutation matrix: 256/256 repeat-stable.
- No mutation result is interpreted as formal fuzzing completeness.

20. Compiler, Analyzer and Sanitizer Gates

The translator is compiled with C11 plus a strict warning set: Wall, Wextra, Wpedantic, Werror, Wshadow, Wstrict-prototypes, Wmissing-prototypes, Wformat=2, Wundef and fno-common. The same source is built with GCC and Clang. The Clang-built translator runs the core corpus independently.

GCC -fanalyzer is run over every translator source file. During finalization this gate found a real cleanup defect: a short-circuit fflush/fclose expression allowed the analyzer to model a path where fclose was not called. The code was changed so flush and close are attempted independently, after which the analyzer passed.

ASan and UBSan build the translator with debug information and frame pointers, then run the core suite. This exercises parser, arena, string buffer, SHA-256 and emission code under memory and undefined-behavior instrumentation. The sanitizer suite passes.

Another strict-Clang issue was fixed rather than suppressed: format-wrapper functions received printf-format attributes so Wformat=2 can reason about call sites while accepting their internal vsnprintf use. The final engineering principle is simple: a failing gate is a bug queue, not a paragraph in the documentation.

- Strict GCC: PASS.
- Strict Clang: PASS.
- GCC analyzer: PASS.
- ASan/UBSan core: PASS.

21. Rust Compilation Gate: Executed vs Shipped

The validation host used to build this release does not contain rustc. Aegis-C2R does not convert that absence into a fake success. The final evidence file records RUSTC_GATE=SKIP and the engineering manual repeats the limitation.

The gate itself is implemented. tools/test-core.sh checks whether rustc is on PATH. If present, each positive and generated Rust source is compiled with Rust 2021 edition and warnings denied. The resulting Rust binary is run and stdout is compared byte-for-byte with the GCC-built C reference.

This design allows the package to be taken to a Rust-equipped Linux host without modifying the tests. A release candidate intended to claim full C-vs-Rust runtime differential evidence should be rerun there and the report regenerated with RUSTC_GATE=PASS.

The current host still validates generated Rust structurally through a deterministic emission guard: forbid(unsafe_code) must exist, unsafe tokens outside that attribute are rejected, delimiters must balance, main lowering must exist, and all checked arithmetic helpers must be present. That is useful but explicitly not mislabeled as rustc syntax/type checking.

- Current release-host rustc gate: SKIP.
- Shipped automatic rustc gate: implemented.
- Do not change the report text to PASS without executing it.

22. Security and Threat Model

The principal threat is malformed or adversarial source causing the translator to crash, emit non-deterministic output, exceed declared resources or generate Rust that means something different from the admitted C profile. The current mitigations are bounded input/AST structures, fail-closed parsing, exact symbol rules, checked output buffers, atomic per-file writes, strict compilers, analyzer, sanitizers and mutation testing.

The translator performs no network access and has no plugin or dynamic-code execution path. It does not run the input C program as part of translation. Runtime tests compile curated/generated fixtures separately under the test harness. This separation is important if the front end is eventually used on untrusted repositories.

The Aegis proof VM has its own verified bytecode boundary and explicit resource model. In this project it executes only translator-generated structural certificate programs, not arbitrary source-derived native code.

Known security limitations remain. This is not an independently audited parser. It is not a sandbox. Per-file atomic output does not provide a cross-filesystem transaction across every requested artifact if the filesystem itself fails mid-write. SHA-256 establishes integrity identity but not authenticity; release signing is future work.

- No network fallback.
- No LLM runtime dependency.
- No generated unsafe Rust block in the declared emitter.
- No claim of independent security certification.

23. Build, Use and Reproduce

The clean source package is designed to be understandable from a terminal. make builds the strict C11 translator. make check bootstraps the vendored Aegis tools and runs the main release corpus. make analyzer, make clang-check, make sanitizer and make aegis-check run the specialized gates.

The check command validates a C file without producing translation artifacts. The ast command prints the canonical AST. The translate command requires an output Rust path and can additionally write an AST, JSON proof and an Aegis certificate prefix that expands into .dxl, .dxs and .dxasm files.

Translation artifacts are created only after the source has passed parsing/profile checks and all in-memory emitters have succeeded. Exact duplicate input/output path strings are rejected to prevent the most direct accidental overwrite cases.

The full verification entry point is tools/final_verify.sh. It cleans, rebuilds, runs every major gate, restores the final optimized translator and prints its SHA-256. A reviewer can start there, then inspect evidence/FINAL_VALIDATION_REPORT.md for the intended interpretation.

```
make
make check
make analyzer
make clang-check
make sanitizer
make aegis-check

build/bin/aegis-c2r translate input.c -o output.rs \
  --ast output.ast --proof proof.json --aegis-prefix certificate
```

24. Git and Release Packaging

Two distinct archives are produced because source review and “everything needed to inspect the tested release” are different use cases. The clean Git archive contains a real initialized repository with one release commit, source, tests, docs, evidence and vendored Aegis source, but excludes build directories and release staging.

The Full AIO archive contains a source snapshot plus prebuilt Linux x86-64 translator binaries, prebuilt Aegis tools, demonstration outputs, documentation and evidence. It is for convenient inspection; the clean Git archive is the better starting point for reproduction.

Both archives are built deterministically by sorting paths, fixing ZIP timestamps, normalizing file permissions and excluding volatile caches. Each archive is built twice during release sealing and the two byte streams must match. SHA-256 files are emitted next to the archives.

A master handoff ZIP contains the clean Git ZIP and the Full AIO ZIP side by side together with their checksums and a README_FIRST file. This matches the practical handoff request: one download, with a reviewer-ready repo and a self-contained evidence/build package inside.

- Clean Git: source-of-truth review.
- Full AIO: compiled conveniences plus source/evidence/docs.
- Master: two inner ZIPs + checksums.

25. Roadmap: From S0 to Repository-Scale Work

The next milestone should not be “accept more syntax at any cost.” It should preserve the evidence model while extending expressiveness. S1 can add fixed-size arrays and bounded indexing with explicit failure semantics. S2 can add structs in a layout-independent mode. Both can remain safe Rust without raw-pointer emulation.

Pointer support is the major architectural step. A credible S3 should use an explicit checked pointer/region abstraction carrying provenance, nullability, bounds and lifetime state rather than pretending arbitrary C pointers are Rust references. That is where published systems such as Cpp2Rust and ownership-analysis work become important comparison points.

Repository-scale S4 needs compilation-database ingestion, exact preprocessing capture, header/content identities and a deterministic dependency scheduler. Validation then expands from stdout comparisons to library APIs and ABI-sensitive behavior.

S5 can integrate optional Kani proof harnesses, Miri runs, C sanitizers, fuzzing and symbolic equivalence for selected kernels. S6 can add repair only after validators are strong enough. Learned/LLM suggestions, if ever added, stay outside the trusted deterministic core and every change must re-enter the complete evidence pipeline.

- Expand evidence before breadth.
- Never promote planned verification to a shipped claim.
- Keep deterministic Aegis witnesses as the invariant through later profiles.

26. Comparison, References and Final Position

C2Rust demonstrates mature C99-oriented migration and cross-checking but intentionally begins with closely mirrored unsafe Rust. Cpp2Rust demonstrates that safe translation of much harder C++ pointer semantics can be approached with a runtime abstraction. Kani and Miri demonstrate complementary Rust verification techniques. Canonical's new program shows that repository scheduling, validation and repair are active industrial research problems.

Aegis-C2R does not need to be broader than these systems to be valuable as a portfolio artifact. Its value is that the first slice is executable, small enough to audit and accompanied by deterministic evidence. The release records defects found during its own finalization and the fixes that made the gates pass.

The professional claim to make is therefore specific: "I built a strict-C deterministic C-to-safe-Rust translator for a documented scalar profile, integrated it with my three-language Aegis bootstrap/toolchain, and built positive, negative, boundary, mutation, sanitizer, analyzer, byte-identity and cross-language certificate tests around it." That claim is supported by files another engineer can run.

The next credibility step is straightforward: rerun the shipped suite on a pinned Rust 1.98.x host so the rustc runtime differential gate changes from SKIP to PASS, then extend the profile one semantics category at a time.

- Canonical: https://discourse.ubuntu.com/t/investing-in-automated-c-to-rust-translation/86369
- C2Rust: https://github.com/immunant/c2rust and https://c2rust.com/manual/
- Cpp2Rust: https://github.com/Cpp2Rust/cpp2rust
- Kani: https://github.com/model-checking/kani
- Miri: https://github.com/rust-lang/miri/